

NumPy Tutorial 2: Creating Arrays — 1D, 2D, and 3D

This chapter covers how to create different types of NumPy arrays. After completing this tutorial, students will understand how to create 1D, 2D, and 3D arrays and use special NumPy functions to generate arrays quickly.

1. Introduction

In the previous tutorial, we learned what NumPy is and how to create a basic array. In this tutorial, we will go deeper and learn how to create different types of arrays.

NumPy arrays can have multiple dimensions. Understanding dimensions is very important because real world data is often stored in 2D or 3D format.

2. What is a Dimension in an Array

A dimension means the number of levels of depth in an array.

- **1D array** — a simple list of numbers (one level)
- **2D array** — rows and columns (two levels — like a table or spreadsheet)
- **3D array** — multiple 2D arrays stacked together (three levels — like a cube)

Think of it like this:

- 1D = a single row of seats in a cinema
 - 2D = the full cinema hall with rows and columns
 - 3D = multiple cinema halls stacked on top of each other
-

3. Creating a 1D Array

A 1D array has only one row of elements.

```
import numpy as np

arr_1d = np.array([10, 20, 30, 40, 50])

print(arr_1d)
print("Dimensions:", arr_1d.ndim)
print("Shape:", arr_1d.shape)
```

Output:

```
[10 20 30 40 50]  
Dimensions: 1  
Shape: (5,)
```

In this example:

- The array has 5 elements in a single row.
- ndim shows 1 — meaning one dimension.
- Shape shows (5,) — meaning 5 elements, no columns.

4. Creating a 2D Array

A 2D array has rows and columns — just like a table.

```
import numpy as np  
  
arr_2d = np.array([[1, 2, 3],  
                  [4, 5, 6],  
                  [7, 8, 9]])  
  
print(arr_2d)  
print("Dimensions:", arr_2d.ndim)  
print("Shape:", arr_2d.shape)
```

Output:

```
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]  
Dimensions: 2  
Shape: (3, 3)
```

In this example:

- We pass a list of lists to create a 2D array.
- Shape shows (3, 3) — meaning 3 rows and 3 columns.
- ndim shows 2 — meaning two dimensions.

5. Creating a 3D Array

A 3D array is a collection of multiple 2D arrays.

```
import numpy as np

arr_3d = np.array([[[1, 2], [3, 4]],
                   [[5, 6], [7, 8]]])

print(arr_3d)
print("Dimensions:", arr_3d.ndim)
print("Shape:", arr_3d.shape)
```

Output:

```
[[[1 2]
  [3 4]]
 [[5 6]
  [7 8]]]
Dimensions: 3
Shape: (2, 2, 2)
```

In this example:

- Shape shows (2, 2, 2) — meaning 2 blocks, 2 rows, 2 columns.
- 3D arrays are used in image processing and deep learning.

6. Special Array Creation Functions

NumPy provides special functions to create arrays quickly without manually typing all values.

np.zeros — Array of all zeros

```
arr = np.zeros((3, 4))
print(arr)
```

Output:

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
```

This creates a 3 row, 4 column array filled with zeros. Very useful when you need an empty array to fill later.

np.ones — Array of all ones

```
arr = np.ones((2, 3))  
print(arr)
```

Output:

```
[[1. 1. 1.]  
 [1. 1. 1.]]
```

np.full — Array filled with a specific value

```
arr = np.full((3, 3), 7)  
print(arr)
```

Output:

```
[[7 7 7]  
 [7 7 7]  
 [7 7 7]]
```

np.eye — Identity Matrix

```
arr = np.eye(4)  
print(arr)
```

Output:

```
[[1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]]
```

An identity matrix has 1s on the diagonal and 0s everywhere else. It is used in mathematics and Machine Learning.

7. Creating Arrays with Ranges

np.arange — Like Python range but for arrays

```
arr = np.arange(0, 20, 2)
print(arr)
```

Output:

```
[ 0  2  4  6  8 10 12 14 16 18]
```

In this example:

- Start: 0
- Stop: 20 (not included)
- Step: 2 (count by 2)

np.linspace — Evenly spaced numbers between two values

```
arr = np.linspace(0, 1, 5)
print(arr)
```

Output:

```
[0.   0.25 0.5  0.75 1.   ]
```

This creates 5 evenly spaced numbers between 0 and 1. Very useful in plotting graphs.

8. Creating Random Arrays

NumPy has a random module to generate random numbers.

Random integers

```
arr = np.random.randint(1, 100, size=(3, 3))
print(arr)
```

This creates a 3x3 array with random integers between 1 and 100.

Random floats between 0 and 1

```
arr = np.random.rand(3, 3)
print(arr)
```

Random arrays are very useful for testing and simulation purposes.

9. Conclusion of Tutorial 2

In this tutorial, students have learned:

1. What dimensions are in NumPy arrays.
2. How to create 1D, 2D, and 3D arrays.
3. How to use `np.zeros`, `np.ones`, `np.full`, and `np.eye`.
4. How to use `np.arange` and `np.linspace` for range arrays.
5. How to create random arrays using `np.random`.

In the next tutorial, we will learn how to perform mathematical operations on arrays — addition, subtraction, multiplication, and more.

This completes NumPy Tutorial 2.